

Week 6 Applied Sheet

Objectives: The applied sessions, in general, give practice in problem solving, in analysis of algorithms and data structures, and in mathematics and logic useful in the above.

Instructions to the class: Work with your peers, and actively participate in the class.

Instructions to TAs: The purpose of the applied class is not to solve the practical exercises! The purpose is to guide students, discuss particular sticking points, and not simply make answers available.

Supplementary problems: The supplementary problems provide additional practice for you to complete after your applied class. Problems that are marked as **(Advanced)** difficulty are beyond the difficulty that you would be expected to complete in this unit, but are nonetheless useful practice problems as they will teach you skills and concepts that you can apply to other problems.

Problems

Problem 1. Consider a buggy implementation of Dijkstra's algorithm in which the distance estimate to a vertex is only updated the first time the vertex is discovered, and not in any subsequent relaxations. Give a graph on which this bug will cause the algorithm to not produce the correct answer.

Problem 2. Can Prim's and Kruskal's algorithms be applied on a graph with negative weights to compute a minimum spanning tree? Why or why not?

Problem 3. Consider the problem of planning a cross-country road trip. You have a map of Australia consisting of the locations of towns, each of which has a petrol station, with the corresponding petrol prices (which may be different at each town). You are currently at a particular town s and would like to travel to town t . Your car has a fuel capacity of C litres, and for each road on the map, you know the amount of petrol it will take to travel along it. Your tank can only contain non-negative integer amounts of petrol, and all roads cost an integer amount of petrol to travel along. You cannot travel along a road if you do not have enough petrol to make it all the way. You may refuel at any petrol station whether your tank is empty or not (but only to integer values), and you are not required to fill your tank. Assuming that your tank is initially empty, describe an algorithm for determining the cheapest way to travel to city t . [Hint: You should model the problem as a shortest path problem and use Dijkstra's algorithm.]

Problem 4. Consider a variant of the shortest path problem where instead of finding paths that minimise the total weight of all edges, we instead wish to minimise the weight of the largest edge appearing on the path. Let's refer to such a path as a *bottleneck path*.

- (a) Give an example of a graph in which a shortest path between some pair of vertices is not a bottleneck (and where the graph also contain a bottleneck path which is not a shortest path).
- (b) Prove that all of the paths in a minimum spanning tree M of a graph G are bottleneck paths in G .
- (c) Prove that not all bottleneck paths of G are paths in a minimum spanning tree M of G .

Problem 5. Consider the following algorithm quite similar to Kruskal's algorithm for producing a minimum spanning tree of a connected graph:

```
1: function MINIMUM_SPANNING_TREE( $G = (V, E)$ )
2:   sort( $E$ , descending, key( $(u, v)$ ) =  $w(u, v)$ )           // Sort edges in order from heaviest to lightest
3:    $T = E$ 
4:   for each edge  $e$  in nonincreasing order do
5:     if  $T - \{e\}$  is connected then
6:        $T = T - \{e\}$ 
```

7: **return** T

Instead of adding edges in order of weight (lightest first) and keeping the graph acyclic, we remove edges in order of weight (heaviest first) while keeping the graph from becoming disconnected.

- (a) Identify a useful invariant maintained by this algorithm.
- (b) Prove that this invariant is maintained by the algorithm.
- (c) Deduce that this algorithm correctly produces a minimum spanning tree.

Problem 6. Recall from the lecture recordings that breadth-first search can be used to find single-source shortest paths on unweighted graphs, or equivalently, graphs where all edges have weight one. Consider the similar problem where instead of only having weight one, edges are now allowed to have weight zero or one. We call this the *zero-one* shortest path problem. Write pseudocode for an algorithm for solving this problem. Your solution should run in $O(V + E)$ time (this means that you cannot use Dijkstra's algorithm!) [Hint: Combine ideas from breadth-first search and Dijkstra's algorithm]

Problem 7. You are the manager of a super computer and receive a set of requests for allocating time frames in that computer. The i -th request specifies the starting time s_i and the finishing time f_i . A subset of requests is compatible if there is no time overlap between any requests in that subset. Develop an algorithm to choose a compatible subset of maximal size (i.e., you want to accept as many requests as feasible, but you cannot select incompatible requests). [Hint: Develop a greedy algorithm.]

Supplementary Problems

Problem 8. For the union-find data structure, there are multiple heuristics to handle how we union two disjoint-set trees. One such heuristic is "union-by-size", where the disjoint-set with less elements is appended to the disjoint set with more elements. We want to prove that the worst-case cost of all the union operations is $O(V \log(V))$ for a disjoint set of V elements, when using this heuristic.

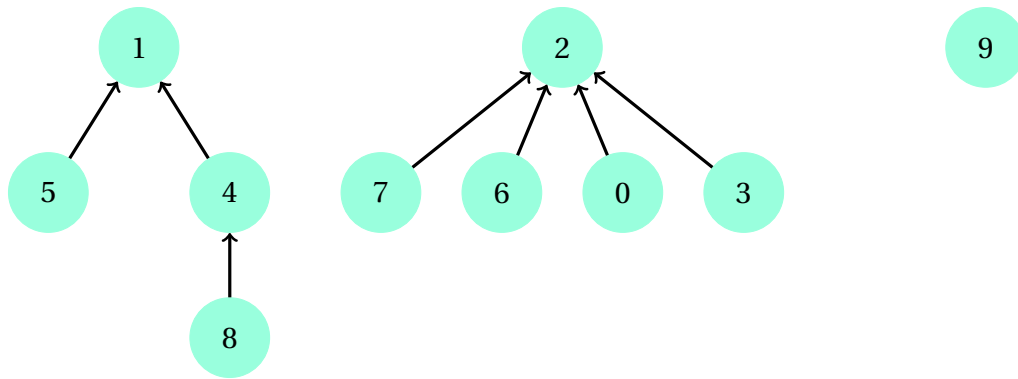
- (a) First prove the following lemma: For any disjoint-set tree constructed via "union-by-size" heuristic, $\text{size}(r) \geq 2^{\text{height}(r)}$, where r represents the root node of some disjoint-set tree, $\text{size}(r)$ represents the number of nodes in the disjoint set tree, and $\text{height}(r)$ represents the height of the tree.
- (b) Using the lemma from part (a), prove that a union takes $O(\log(V))$.
- (c) Hence, prove that all the unions take $O(V \log(V))$.

Problem 9. Suppose that we wish to solve the single-source shortest path problem for graphs with non-negative bounded integer edge weights, i.e. there is a constant c such that for all edges (u, v) it holds that $0 \leq w(u, v) \leq c$. Explain how we can modify Dijkstra's algorithm to run in $O(V + E)$ time in this case. [Hint: Improve the priority queue]

Problem 10. Consider the following supposed invariant for Kruskal's algorithm: At each iteration, each connected component in the current spanning forest is a minimum spanning tree of the vertices in that component.

- (a) Prove or disprove whether Kruskal's algorithm maintains this invariant.
- (b) Can this invariant be used to prove that Kruskal's algorithm is correct?

Problem 11. Consider the following state of a union-find data structure:



- (a) Perform the following `union()` calls sequentially using the *union-by-rank* and *path compression* heuristics, visualising the outcome.
- (i) `union(4, 0)`
 - (ii) `union(8, 3)`
 - (iii) `union(9, 0)`
- (b) Compare the difference between the final state of the forest and the state of the forest if you were to use the *union-by-size* heuristic alone.